Local-First Agentic Content Intelligence (L-ACI)

Project Specification & MLOps Blueprint

1. Executive Summary

The **L-ACI Platform** is a self-hosted, sovereign multi-agent system designed for automated content discovery, quantification, and curation. Developed for an ML Engineer's portfolio, this project emphasizes **LLMOps, Local Inference, and Cloud-Native Orchestration**.

By leveraging **Kubernetes (K3s/Minikube)** and **Ollama**, the system avoids 3rd-party API costs while providing a production-grade environment for managing agentic workflows and feedback-driven personalization.

2. Technical Stack

Layer	Technology	Role
Orchestration	Kubernetes (K3s) / Helm	Cluster management & GPU scheduling
Inference	Ollama (Llama 3.1 / Mistral)	Local LLM serving
Backend	FastAPI	Asynchronous API & Orchestrator
Agents	LangGraph	Stateful multi-agent reasoning
Database	PostgreSQL + pgvector	Metadata & Vector storage
Task Queue	Redis	Asynchronous scraping & processing
Search	SearXNG	Private, local search engine

Observability	Arize Phoenix	Tracing & Evaluation

3. System Architecture

The system is designed as a microservices architecture deployed within a local Kubernetes cluster.

3.1 Component Breakdown

- **API Gateway (FastAPI):** Handles user requests, triggers ingestion, and serves the curated daily digest.
- **Agent Swarm (LangGraph):** A cyclic graph of agents that scrape, evaluate, and summarize content.
- **Vector Memory:** Stores the user's "Interest Profile" and past content embeddings to ensure novelty.
- **GPU-Accelerated Inference:** Ollama pods utilizing the NVIDIA Device Plugin to leverage local hardware.

3.2 Agent Workflow Logic

The agentic flow uses a 4-node structure in LangGraph:

1. **The Scout:** Polls RSS, ArXiv, and SearXNG. Cleans HTML to markdown.
2. **The Critic:** Embeds the content and calculates a relevance score against the user profile.
3. **The Researcher:** If score > threshold, it searches the Vector DB for historical context.
4. **The Editor:** Synthesizes the information into a structured report.

4. ML & Engineering Mathematics

4.1 Importance Quantification

Importance is determined by a weighted multi-objective function:

$$S_{total} = w_1 \cdot Sim(v_{art}, v_{user}) + w_2 \cdot Novelty + w_3 \cdot Authority$$

Where Sim is the Cosine Similarity between the article embedding and the user interest vector.

4.2 The Feedback Loop (Active Learning)

User feedback updates the profile using an **Exponential Moving Average (EMA)** to allow the system to adapt to changing interests:

$$v_{user,\ t+1} = (1 - \alpha)v_{user,\ t} + \alpha v_{feedback}$$

5. Kubernetes Orchestration Details

The deployment is managed via Kubernetes manifests. Key MLOps configurations include:

> **GPU Scheduling:** The Ollama pod requires specific resource requests to access the local GPU:
>
> ```
> resources:
>   limits:
>     nvidia.com/gpu: 1
> ```

5.1 Stateful Storage

To ensure persistence across pod restarts, PostgreSQL and Ollama models are stored on **Persistent Volumes (PV)** using the *local-path* provisioner.

6. Implementation Roadmap

Phase 1: Local Backbone (Week 1)

- Dockerize FastAPI and PostgreSQL with pgvector.
- Implement the **Scout Agent** using Playwright.
- Setup **Ollama** locally and test basic inference.

Phase 2: Kubernetes Migration (Week 2)

- Setup **K3s** or **Minikube**.
- Deploy Redis and Postgres via Helm.
- Configure the NVIDIA Device Plugin for GPU access.

Phase 3: Agentic Intelligence (Week 3)

- Develop the **LangGraph** state machine.

- Implement **SearXNG** for internal research capabilities.
- Integrate **Arize Phoenix** for local tracing.

Phase 4: Feedback & Optimization (Week 4)

- Build the `/feedback` endpoint for vector updates.
- Benchmark model performance (Tokens/Sec vs Quantization level).